

RePlot

Land Readjustment & Replotting Tool

Complete Architectural Analysis, Missing Components & Implementation Roadmap

Version 1.0 — March 2026

Table of Contents

Table of Contents	2
1. Executive Summary	4
1.1 What Your Application Already Does Well	4
1.2 Critical Gaps (What Is Missing)	4
2. Current Architecture Map	5
2.1 Project Structure	5
2.2 Dependency Flow (Current)	5
3. Critical Performance Issues	6
3.1 R-Tree Rebuilds on Every Insert (SEVERE).....	6
What To Do	6
3.2 GDI+ Pen/Brush Allocation in Draw Loops	6
What To Do	6
3.3 Mouse Move Queries Entire Visible World	6
What To Do	7
4. Missing Object-Oriented Patterns	8
4.1 No Dependency Injection (DI)	8
What Exists Now	8
What Should Exist.....	8
4.2 No Custom Events/Delegates for Cross-Component Communication	8
Implementation Pattern (C# Events + Delegates)	8
4.3 No Service Layer (Business Logic Mixed with UI)	9
What Should Exist.....	9
5. Missing Domain Concepts for Land Replotting	10
5.1 Missing: Parcel Entity	10
What Should Exist.....	10
5.2 Missing: Layer System	10
What Should Exist.....	10
5.3 Missing: Topology Validation.....	11
5.4 Missing: Coordinate Reference System (CRS).....	11
6. Asynchronous Operations, Error Handling & Logging	12
6.1 No Async/Await Anywhere	12
What Should Exist.....	12
6.2 Error Handling Is Ad-Hoc	12
What Should Exist.....	12
6.3 No Global Exception Handler	13
7. Database Architecture & ORM.....	14
7.1 Current State	14

7.2 Why Not Entity Framework (Yet).....	14
7.3 Repository Pattern (Implement This First)	14
7.4 Transaction Support.....	14
8. Canvas Rendering — Deep Dive for High-Zoom Quality	16
8.1 Current Rendering Pipeline.....	16
8.2 High-Zoom Rendering Issues.....	16
What You Already Do Right.....	16
What Is Missing.....	16
8.3 Zoom-Dependent Line Width	16
8.4 Grid Rendering at Extreme Zoom	16
9. Complete Gap Analysis.....	17
10. Implementation Roadmap.....	18
Phase 1: Foundation Fixes (Do First — 1-2 weeks)	18
Phase 2: Domain Model (Weeks 2-4)	18
Phase 3: Persistence (Weeks 4-6).....	18
Phase 4: Replotting Operations (Weeks 6-10)	18
Phase 5: Polish (Weeks 10-12).....	18
11. Recommended Target Architecture.....	20
11.1 Clean Architecture Layers	20
11.2 Folder Structure (Target).....	20
12. Quick Wins (Fix Today).....	21
1. Remove the duplicate SetStyle calls	21
2. Fix the R-Tree with a dirty flag.....	21
3. Remove MessageBox from shape Draw methods.....	21
4. Add the viewoffsetX/Y cleanup.....	21
5. Seal the App.config issue	21
13. Technology Decision Matrix.....	22
14. Conclusion.....	23

(Update this field in Word: right-click → Update Field)

1. Executive Summary

This document is a complete architectural analysis of the RePlot application — a specialized land readjustment and replotting tool built in C# with WinForms (.NET 10). The application is NOT a generic CAD or GIS tool; it is purpose-built for land pooling workflows in Nepal (parcel creation, division, movement, contribution calculation, and owner management).

The analysis covers every source file in the project, identifies what has been done well, what is missing, and provides a concrete implementation roadmap. The goal is to help you understand each architectural decision so you can learn and build independently.

1.1 What Your Application Already Does Well

- **Interface segregation:** IShape defines a clean contract; concrete shapes (Line, Rectangle, Polyline, Circle, Ellipse) implement it independently. This is textbook OOP.
- **Spatial indexing:** RTreeSpatialIndex wraps NetTopologySuite STRtree for viewport culling. This is industry-standard for GIS and CAD.
- **Command pattern:** UndoRedoManager + ICommand gives you memory-efficient undo/redo (stores deltas, not full state snapshots).
- **Deferred rendering:** OptimizedDeferredRenderer uses bitmap caching and pan-buffer blitting — the same technique QGIS and MapInfo use.
- **Coordinate system:** DrawingEngine correctly handles Y-axis flip (screen Y goes down, world Y goes up) with double-precision math.
- **Object snapping:** SnapManager computes endpoints, midpoints, intersections, perpendiculars — matching AutoCAD snap behavior.

1.2 Critical Gaps (What Is Missing)

Despite the solid foundation, the application has significant gaps that will prevent it from scaling to real-world use. The sections that follow analyze each gap in detail.

2. Current Architecture Map

2.1 Project Structure

The solution contains two projects: the main Land_Readjustment_Tool (WinExe, .NET 10 WinForms) and an Area Converter library. The main project is organized into these major folders:

Layer	Folder	Key Files	Responsibility
UI / Forms	Forms/	frmMain, frmImportParcelOwnership, frmLandParcelOwnersRecord	User interaction, menu routing, data display
Canvas Control	MapCanvas/	DrawingCanvasControl (UserControl)	2D workspace, toolbar, mouse events, paint
Core Engine	MapCanvas/Core/	DrawingEngine, ShapeManager	Coordinate transforms, spatial queries
Rendering	MapCanvas/Rendering/	CanvasRenderer, OptimizedDeferredRenderer	Grid, shapes, overlays, bitmap cache
Shapes	MapCanvas/Models/Shapes/	IShape, Shape, LineShape, PolylineShape, etc.	Geometric primitives, drawing, hit-testing
Snapping	MapCanvas/Models/Snapping/	SnapManager, SnapPoint, ISnapProvider	Object snap engine
Commands	MapCanvas/Core/Commands/	ICommand, UndoRedoManager, ShapeCommands	Undo/redo via command pattern
Spatial Index	MapCanvas/Core/Spatial Index/	RTreeSpatialIndex	NTS STRtree wrapper
Data	MapCanvas/Data/	ShapeRepository (stub)	Persistence layer (NOT implemented)
Services	Services/	DatabaseHelper, DatabaseSchema, ExcellImport, etc.	Business logic, data access

2.2 Dependency Flow (Current)

Currently, dependencies flow downward but are tightly coupled. DrawingCanvasControl directly creates DrawingEngine, ShapeManager, CanvasRenderer, and OptimizedDeferredRenderer in its constructor. There is no Dependency Injection container, no interface abstraction for the engine or renderer, and no service layer between the UI and database.

```

DrawingCanvasControl → new DrawingEngine(size)
DrawingCanvasControl → new ShapeManager(bounds)
DrawingCanvasControl → new CanvasRenderer()
DrawingCanvasControl → new OptimizedDeferredRenderer(size, engine, shapeManager)

```

This means you cannot unit test any component in isolation, cannot swap implementations, and cannot share state between multiple views.

3. Critical Performance Issues

3.1 R-Tree Rebuilds on Every Insert (SEVERE)

This is the single most critical performance bug in the application. Look at `RTreeSpatialIndex.Insert()`:

```
public void Insert(IShape shape) {
    _allShapes.Add(shape);
    Rebuild(); // <-- FULL REBUILD every single insert!
}
```

STRtree is a bulk-loaded, immutable index. Once built, you cannot add items. Your current code works around this by rebuilding the entire tree on every single insertion. For 10,000 shapes, this means 10,000 full rebuilds during a DXF import — $O(n^2)$ complexity.

What To Do

Replace the naive rebuild strategy with a two-phase approach:

- **Dirty flag:** Mark the index as dirty on insert/remove. Do NOT rebuild immediately.
- **Deferred rebuild:** Rebuild only when a query is requested and the index is dirty. This is called lazy evaluation.
- **Bulk loading:** For imports (100+ shapes), collect all shapes first, then call `Rebuild()` once at the end. STRtree is optimized for bulk loading.

```
public void Insert(IShape shape) {
    _allShapes.Add(shape);
    _isDirty = true; // Just mark dirty
}

public List<IShape> Query(RectangleD area) {
    if (_isDirty) { Rebuild(); _isDirty = false; }
    // ... actual query
}
```

This single change will improve DXF import from $O(n^2)$ to $O(n \log n)$ — a 1000x speedup for 10,000 shapes.

3.2 GDI+ Pen/Brush Allocation in Draw Loops

Every shape creates new Pen and Brush objects inside its `Draw()` method using 'using' blocks. For 5,000 shapes rendered 60 times per second, that is 300,000 GDI+ object allocations per second. Each allocation involves a kernel call to the OS.

What To Do

Implement a `PenCache/BrushCache` that pools pens by (color, width) tuple. `CanvasRenderer` already pools grid pens correctly — extend this pattern to all shapes. Pass a `RenderContext` object into `Draw()` that contains pre-allocated pens for normal, selected, and preview states.

3.3 Mouse Move Queries Entire Visible World

On every mouse move, the snap system queries all shapes in the visible world rectangle. At zoom-out levels showing thousands of parcels, this means testing every single shape for snap proximity on every pixel the mouse moves.

What To Do

Query a small region around the mouse cursor (e.g., 50 world-units radius) instead of the entire viewport. You already have the spatial index for this — just change the query rectangle from `GetVisibleWorldRectangle()` to a small box around the mouse position.

4. Missing Object-Oriented Patterns

4.1 No Dependency Injection (DI)

Your DrawingCanvasControl creates all its dependencies with 'new'. This is the opposite of Dependency Injection. In professional applications like ETABS or AutoCAD, components are wired together through a DI container so each piece can be tested, replaced, or configured independently.

What Exists Now

```
// Inside DrawingCanvasControl constructor:
_engine = new DrawingEngine(panelCanvas.Size);
_shapeManager = new ShapeManager(worldBounds);
_renderer = new CanvasRenderer();
```

What Should Exist

Define interfaces (IDrawingEngine, IShapeManager, ICanvasRenderer) and inject them. For WinForms without a full DI container, use a simple ServiceLocator or Manual DI at the composition root (Program.cs or frmMain).

```
// Interfaces:
public interface IDrawingEngine { PointD WorldToScreen(PointD p); ... }
public interface IShapeManager { void AddShape(IShape s); ... }

// Composition root (frmMain):
var engine = new DrawingEngine(size);
var shapeManager = new ShapeManager(bounds);
var renderer = new CanvasRenderer();
canvasControl.Initialize(engine, shapeManager, renderer);
```

4.2 No Custom Events/Delegates for Cross-Component Communication

Currently, DrawingCanvasControl exposes only one event: CollapseLeftPanelClicked. A replotting application needs rich event communication:

Event	Publisher	Subscriber(s)	Purpose
ShapeAdded / ShapeRemoved	ShapeManager	Canvas, PropertyPanel, Database	Refresh display, update properties, auto-save
SelectionChanged	ShapeManager or Canvas	PropertyPanel, StatusBar	Show selected parcel info
ViewportChanged	DrawingEngine	MiniMap, Renderer, StatusBar	Update overview, refresh cache
LayerVisibilityChanged	LayerManager (missing)	Canvas, Legend	Toggle layer display
ToolChanged	Toolbar/Canvas	StatusBar, CursorManager	Update UI feedback
ParcelSplit / ParcelMerge	ReplotService (missing)	Database, Canvas, Reports	Core replotting operations
UndoRedoStateChanged	UndoRedoManager	Toolbar (enable/disable buttons)	Update button states

Implementation Pattern (C# Events + Delegates)

```
// In ShapeManager:
```



```

public event EventHandler<ShapeEventArgs> ShapeAdded;
public event EventHandler<ShapeEventArgs> ShapeRemoved;
public event EventHandler<SelectionChangedEventArgs> SelectionChanged;

public void AddShape(IShape shape) {
    _shapes.Add(shape);
    _spatialIndex.Insert(shape);
    ShapeAdded?.Invoke(this, new ShapeEventArgs(shape));
}

```

4.3 No Service Layer (Business Logic Mixed with UI)

DXF import logic lives directly inside `DrawingCanvasControl.btnImportDxf_Click`. Database save logic is scattered across form code-behind files. There is no service layer to encapsulate business rules.

What Should Exist

A Services layer that encapsulates all business operations. The UI layer should only call service methods — it should never directly manipulate data or perform calculations.

```

// Services/IDxfImportService.cs
public interface IDxfImportService {
    Task<ImportResult> ImportAsync(string filePath, IProgress<int> progress);
}

// Services/IReplotService.cs
public interface IReplotService {
    SplitResult SplitParcel(IShape parcel, PointD splitStart, PointD splitEnd);
    MergeResult MergeParcel(IShape parcel1, IShape parcel2);
    double CalculateContribution(IShape originalParcel, double contributionRate);
}

```

5. Missing Domain Concepts for Land Replotting

This is the most important section. Your application draws lines, rectangles, circles, and polylines. But land replotting requires specific domain objects that do not exist yet.

5.1 Missing: Parcel Entity

A Parcel is NOT a PolylineShape. It is a domain object that contains a closed polygon geometry PLUS land-specific metadata. Currently, your PolylineShape has IsClosed and a generic Properties dictionary, but there is no typed Parcel class with validated fields.

What Should Exist

```
public class Parcel : Shape {
    public string ParcelNumber { get; set; }
    public int LandOwnerId { get; set; }
    public List<PointD> Boundary { get; } // Closed polygon
    public double Area => CalculateArea(); // Shoelace formula
    public double Perimeter => CalculatePerimeter();
    public ParcelType Type { get; set; } // Original, Replotted, Road, OpenSpace
    public string ZoningCode { get; set; }
    public double ContributionRate { get; set; }
    public double ContributedArea { get; set; }
    public double NetArea => Area - ContributedArea;
}
```

5.2 Missing: Layer System

IShape has a LayerName string property, but there is no LayerManager class. AutoCAD and QGIS both have a proper layer system with visibility toggles, lock states, default colors, and ordering. Your replotting tool needs at minimum: Original Parcels, Replotted Parcels, Roads, Open Space, Project Boundary, and Annotations.

What Should Exist

```
public class Layer {
    public string Name { get; set; }
    public bool IsVisible { get; set; }
    public bool IsLocked { get; set; }
    public Color DefaultColor { get; set; }
    public int DrawOrder { get; set; }
    public float DefaultLineWidth { get; set; }
}

public class LayerManager {
    public event EventHandler<LayerEventArgs> LayerChanged;
    public IReadOnlyList<Layer> Layers { get; }
    public void AddLayer(Layer layer);
    public void ToggleVisibility(string name);
    public Layer GetLayer(string name);
}
```

5.3 Missing: Topology Validation

Land parcels must form a valid topology — no overlaps, no gaps between adjacent parcels, boundaries must align. Currently, shapes are independent objects with no topological relationships. NetTopologySuite (already in your packages) provides all the tools for this.

5.4 Missing: Coordinate Reference System (CRS)

Your DrawingEngine hardcodes Nepal UTM zone bounds. But there is no CRS management. ProjNET (already in your packages) can handle coordinate transformations. You need a CRS manager so that imported DXF files in different coordinate systems can be reprojected to match.

6. Asynchronous Operations, Error Handling & Logging

6.1 No Async/Await Anywhere

The entire application runs synchronously on the UI thread. DXF import, database operations, Excel reading — everything blocks the UI. For a file with 50,000 entities, the application will appear frozen.

What Should Exist

```
// In IDxfImportService:
public async Task<ImportResult> ImportAsync(string path, IProgress<int> progress,
CancellationToken ct) {
    var dxf = await Task.Run(() => DxfDocument.Load(path), ct);
    int total = dxf.Entities.All.Count();
    int processed = 0;
    var shapes = new List<IShape>();

    await Task.Run(() => {
        foreach (var entity in dxf.Entities.All) {
            ct.ThrowIfCancellationRequested();
            shapes.Add(ConvertEntity(entity));
            progress?.Report(++processed * 100 / total);
        }
    }, ct);

    _shapeManager.BulkAddShapes(shapes); // Single rebuild
    return new ImportResult(shapes.Count);
}
```

Key principle: heavy computation runs on `Task.Run()`, progress is reported via `IProgress<T>`, cancellation is supported via `CancellationToken`. The UI remains responsive throughout.

6.2 Error Handling Is Ad-Hoc

Current error handling uses `MessageBox.Show()` inside catch blocks within shape `Draw()` methods. This is dangerous because a rendering error in one shape will show a dialog every frame (60 times per second). Additionally, errors in `Draw()` should never crash the application — they should be silently logged and the shape skipped.

What Should Exist

A centralized logging system. For a desktop WinForms application, use a simple `ILogger` interface backed by file logging. Do not add heavyweight frameworks like Serilog unless you need structured logging to a server.

```
public interface IAppLogger {
    void Info(string message);
    void Warn(string message);
    void Error(string message, Exception ex = null);
}

public class FileLogger : IAppLogger {
```

```

    private readonly string _logPath;
    public void Error(string msg, Exception ex) {
        File.AppendAllText(_logPath,
            $"{DateTime.Now:O} [ERROR] {msg} {ex}\n");
    }
}

// In Shape.Draw():
try { /* render */ }
catch (Exception ex) {
    _logger?.Error($"Draw failed for {Id}", ex);
    // Do NOT show MessageBox – silently skip this frame
}

```

6.3 No Global Exception Handler

If an unhandled exception occurs anywhere, the application crashes without saving. Your Program.cs should install global exception handlers.

```

Application.ThreadException += (s, e) => {
    logger.Error("UI thread exception", e.Exception);
    // Auto-save project, show recovery dialog
};
AppDomain.CurrentDomain.UnhandledException += (s, e) => {
    logger.Error("Fatal", (Exception)e.ExceptionObject);
};

```

7. Database Architecture & ORM

7.1 Current State

DatabaseHelper opens a raw SQLiteConnection and creates tables with hand-written SQL strings. ShapeRepository is a complete stub (every method throws NotImplementedException). There is no ORM, no repository pattern with proper async, and no transaction management.

7.2 Why Not Entity Framework (Yet)

Entity Framework Core does support SQLite and spatial types via NTS. However, for a desktop application that works with .rplot project files (each file is a separate SQLite database), EF Core adds complexity. The recommended approach for your current stage:

- **Phase 1 (Now):** Use Dapper (micro-ORM) with raw SQL for simple CRUD. It maps query results to your C# objects with almost zero overhead. This gives you the speed of raw ADO.NET with the convenience of object mapping.
- **Phase 2 (When multi-user needed):** Migrate to EF Core with SQLite provider and NTS spatial. This gives you change tracking, migrations, and LINQ queries.

7.3 Repository Pattern (Implement This First)

```
public interface IParcelRepository {
    Task<Parcel> GetByIdAsync(Guid id);
    Task<List<Parcel>> GetAllAsync();
    Task<List<Parcel>> GetInViewAsync(RectangleD viewport);
    Task InsertAsync(Parcel parcel);
    Task UpdateAsync(Parcel parcel);
    Task DeleteAsync(Guid id);
    Task<List<Parcel>> GetByLayerAsync(string layerName);
    Task<List<Parcel>> FindAdjacentAsync(Guid parcelId);
}
```

Every method is async. The repository handles serialization of geometry to WKB (Well-Known Binary) using NetTopologySuite. The caller (service layer) never sees SQL strings.

7.4 Transaction Support

When splitting a parcel (delete original, create two new parcels), all three operations must succeed or fail together. Wrap them in a transaction:

```
public async Task SplitParcelAsync(Guid originalId, Parcel left, Parcel right) {
    using var transaction = _connection.BeginTransaction();
    try {
        await DeleteAsync(originalId);
        await InsertAsync(left);
        await InsertAsync(right);
        transaction.Commit();
    } catch {
        transaction.Rollback();
        throw;
    }
}
```

}

8. Canvas Rendering — Deep Dive for High-Zoom Quality

8.1 Current Rendering Pipeline

Your `panelCanvas_Paint` method renders 8 layers in order: Background, Grid, Axis markers, Shapes (cached or pan-blit), Preview shape, Polyline preview, Snap glyph, UI overlay, and optionally a snap-disabled warning. This layered approach is correct and matches how professional CAD renderers work.

8.2 High-Zoom Rendering Issues

At extreme zoom levels ($\text{ZoomScale} > 10,000$), GDI+ float precision (7 significant digits) becomes inadequate for world coordinates like UTM Easting 380,000. The product $380,000 * 10,000 = 3.8$ billion exceeds float range, causing visual artifacts.

What You Already Do Right

Your shapes compute in double precision (`PointD`) and only cast to float at the final `DrawLine/DrawRectangle` call. The rounding to nearest pixel (`Math.Round`) also helps. These are correct decisions.

What Is Missing

You need coordinate clamping BEFORE the GDI+ calls. If a screen coordinate exceeds $\pm 1,000,000$ pixels, clamp it. GDI+ handles coordinates within this range without artifacts. Add a `ClampToGDI` helper:

```
private static float ClampToGDI(double v) {
    const double MAX = 1_000_000;
    if (v > MAX) return (float)MAX;
    if (v < -MAX) return (float)-MAX;
    return (float)v;
}
```

8.3 Zoom-Dependent Line Width

Currently, all shapes use a fixed 0.25f pen width. At high zoom this renders as a hairline barely visible against the grid. At low zoom, shapes are hard to distinguish. Professional CAD tools use zoom-dependent line widths:

```
float GetAdaptiveLineWidth(double zoomScale) {
    if (zoomScale > 100) return 1.0f;    // High zoom: visible
    if (zoomScale > 10)  return 0.5f;    // Medium zoom
    return 0.25f;        // Low zoom: thin
}
```

8.4 Grid Rendering at Extreme Zoom

Your adaptive grid with hysteresis (`MIN_MINOR_PIXELS`, `MAX_MINOR_PIXELS`) is a good approach. However, the early return at $\text{zoomScale} < 0.001$ means the grid disappears at very low zoom levels where it is still useful. Consider rendering only major grid lines at low zoom instead of hiding entirely.

9. Complete Gap Analysis

This table lists every identified gap, its severity, the component it affects, and what needs to be done.

#	Gap	Severity	Component	Action Required
1	R-Tree rebuilds on every insert	CRITICAL	RTreeSpatialIndex	Add dirty flag + lazy rebuild
2	No DI container / interfaces for core	HIGH	All Core	Extract interfaces, inject via constructor
3	No Parcel domain entity	CRITICAL	Models	Create Parcel class with area, owner, zoning
4	No Layer system	HIGH	Models/Core	Create Layer + LayerManager classes
5	ShapeRepository is a stub	CRITICAL	Data	Implement with SQLite + WKB serialization
6	No async operations	HIGH	All	Add async to import, save, query operations
7	No events for cross-component comms	HIGH	Core	Add events: ShapeAdded, SelectionChanged, etc.
8	No error logging	HIGH	Program.cs	Add IAppLogger + FileLogger + global handler
9	GDI+ pen allocation in draw loop	MEDIUM	Shapes	Implement PenCache/RenderContext
10	No topology validation	HIGH	Core (new)	Use NTS for overlap/gap detection
11	No CRS management	MEDIUM	Core (new)	Add CRS class using ProjNET
12	DXF import in UI code-behind	MEDIUM	Canvas	Extract to IDxfImportService
13	No auto-save / crash recovery	HIGH	Services (new)	Periodic auto-save to temp file
14	Mouse snap queries full viewport	MEDIUM	SnapManager	Query small region around cursor
15	No MoveShapeCommand	MEDIUM	Commands	Add MoveShapeCommand for drag operations
16	No selection manager (multi-select)	HIGH	Core (new)	Add SelectionManager with rubber-band
17	Duplicate SetStyle calls	LOW	Canvas	Remove duplicate DoubleBuffered setup
18	No unit tests	HIGH	All	Add test project for engine, shapes, commands
19	Duplicate PolylineShape creation	LOW	Canvas	Refactor CreatePreviewShape + FinishDrawing
20	No progress reporting for imports	MEDIUM	Import forms	Use IProgress<T> pattern

10. Implementation Roadmap

Prioritized into phases. Each phase builds on the previous and delivers a working application.

Phase 1: Foundation Fixes (Do First — 1-2 weeks)

These are blocking issues. Fix them before adding any new features.

- **Fix R-Tree lazy rebuild:** Change Insert() to set `_isDirty` flag. Add EnsureBuilt() before Query(). This alone fixes DXF import performance.
- **Extract IDrawingEngine, IShapeManager interfaces:** Do not change any logic — just extract interfaces from existing classes. This enables future DI and testing.
- **Add IAppLogger + FileLogger:** Wire into Program.cs with global exception handlers. Replace all MessageBox.Show in catch blocks with logger calls.
- **Remove duplicate SetStyle/DoubleBuffered calls:** InitializeDrawingSystem() repeats what the constructor already does.
- **Fix Pen allocation:** Create PenCache class. Pass it through RenderContext into shape Draw() methods.

Phase 2: Domain Model (Weeks 2-4)

- **Create Parcel class:** Extends Shape, adds ParcelNumber, OwnerId, Area (Shoelace formula), Type enum, ContributionRate.
- **Create Layer + LayerManager:** With events for visibility changes. Wire into CanvasRenderer to respect draw order.
- **Create SelectionManager:** Manages selected shapes, fires SelectionChanged events, supports rubber-band selection.
- **Add events to ShapeManager:** ShapeAdded, ShapeRemoved, SelectionChanged — so PropertyPanel and StatusBar can subscribe.

Phase 3: Persistence (Weeks 4-6)

- **Implement IParcelRepository with SQLite:** Use Dapper or raw ADO.NET. Serialize geometry as WKB via NTS.
- **Implement project save/load for canvas:** Save all shapes to SQLite when project saves. Load them when project opens.
- **Add auto-save:** Timer-based (every 5 minutes) to a temp file. Recover on crash.
- **Transaction support:** Wrap multi-step operations (split, merge) in transactions.

Phase 4: Replotting Operations (Weeks 6-10)

- **IReplotService:** SplitParcel, MergeParcel, CalculateContribution, AllocateReplot.
- **Topology validation:** No overlaps, no gaps, boundary alignment using NTS.
- **Split tool:** Draw a line across a parcel to split it into two parcels.
- **Contribution calculator:** Deduct percentage from each parcel based on rules.

Phase 5: Polish (Weeks 10-12)

- **Async all I/O:** Import, save, export — all with progress bars and cancellation.
- **Unit tests:** For DrawingEngine transforms, ShapeManager queries, command undo/redo.
- **CRS management:** Detect CRS from DXF import, reproject if needed.

11. Recommended Target Architecture

11.1 Clean Architecture Layers

Layer	Contains	Depends On	Rule
Presentation (UI)	Forms, UserControls, Toolbars	Application layer only	Never touches database or geometry math directly
Application (Services)	DxfImportService, ReplotService, ProjectService	Domain + Interfaces	Orchestrates use cases, calls repositories
Domain (Models)	Parcel, Layer, IShape, PointD, commands	Nothing (pure)	No dependencies on UI, database, or framework
Infrastructure	SqliteParcelRepository, FileLogger, DxfReader	Domain interfaces	Implements interfaces defined in Domain

11.2 Folder Structure (Target)

```

Land_Readjustment_Tool/
├── Domain/
│   ├── Models/           (Parcel, Layer, IShape, PointD, RectangleD)
│   ├── Interfaces/       (IParcelRepository, IAppLogger, IDrawingEngine)
│   ├── Commands/        (ICommand, AddShapeCommand, SplitParcelCommand)
│   └── Events/           (ShapeEventArgs, SelectionChangedEventArgs)
├── Application/
│   ├── Services/         (ReplotService, DxfImportService, ProjectService)
│   └── ViewModels/        (ParcelViewModel – if you add data binding)
├── Infrastructure/
│   ├── Persistence/      (SqliteParcelRepository, DatabaseHelper)
│   ├── Logging/          (FileLogger)
│   └── Import/            (DxfReader, ShapefileReader)
├── Presentation/
│   ├── Forms/            (frmMain, frmImport, frmRecords)
│   ├── Canvas/           (DrawingCanvasControl, CanvasRenderer)
│   └── Controls/          (LayerPanel, PropertyPanel)
└── Program.cs             (Composition root – wire DI here)

```

12. Quick Wins (Fix Today)

These are small changes you can make right now that will immediately improve quality:

1. Remove the duplicate `SetStyle` calls

In `DrawingCanvasControl`, the constructor calls `SetStyle` and sets `DoubleBuffered`. Then `InitializeDrawingSystem()` does the exact same thing again. Delete the duplicate block in `InitializeDrawingSystem()`.

2. Fix the R-Tree with a dirty flag

In `RTreeSpatialIndex`, add a private `bool _isDirty = false` field. In `Insert()` and `Remove()`, set `_isDirty = true` instead of calling `Rebuild()`. In `Query()`, add: `if (_isDirty) { Rebuild(); _isDirty = false; }` at the top.

3. Remove `MessageBox` from shape `Draw` methods

In `RectangleShape.Draw()`, there is a catch block with `MessageBox.Show`. Replace with `Debug.WriteLine` or a logger call. A `MessageBox` in a paint loop will lock the application.

4. Add the `viewoffsetX/Y` cleanup

`DrawingEngine` has public float `viewoffsetX` and `viewoffsetY` properties that are never used (the actual offset is the private `PointD _viewOffset`). Remove these unused properties to avoid confusion.

5. Seal the `App.config` issue

Your `App.config` contains a `<PropertyGroup>` XML element that belongs in `.csproj`, not in `App.config`. This may cause runtime warnings. Remove the `PropertyGroup` from `App.config`.

13. Technology Decision Matrix

Concern	Current	Recommended	Why
ORM	None (raw SQL strings)	Dapper (now), EF Core (later)	Dapper: zero overhead, maps to objects. EF Core: when you need migrations & change tracking
Logging	MessageBox.Show	IAppLogger + FileLogger	Non-blocking, persistent, searchable
DI	None (new everywhere)	Manual DI at composition root	Full DI containers (Autofac) are overkill for WinForms
Spatial	NTS STRtree (correct)	Keep NTS, add lazy rebuild	NTS is industry standard, already in packages
Rendering	GDI+ (correct for WinForms)	Keep GDI+, add PenCache	SkiaSharp only if you hit GDI+ limits (100K+ shapes)
CRS	Hardcoded Nepal UTM	ProjNET (already in packages)	Enables import from any coordinate system
Testing	None	xUnit + Moq	Test engine transforms, spatial queries, commands
Async	None	Task.Run + IProgress<T>	Keeps UI responsive during import/save

14. Conclusion

Your application has a solid foundation. The coordinate system, spatial indexing, command pattern, deferred rendering, and snapping system are all architecturally sound and follow CAD/GIS industry patterns. The critical gaps are not in the algorithms — they are in the software engineering practices around those algorithms.

The three highest-impact changes are: (1) fixing the R-Tree rebuild-on-every-insert bug, (2) creating a proper Parcel domain entity with area calculation and owner linkage, and (3) implementing the repository pattern so your canvas shapes persist to the project database. Everything else — DI, events, logging, async — improves quality but is not blocking your core workflow.

Follow the roadmap in Phase order. Each phase produces a working application. Do not try to implement everything at once. Build incrementally, test each piece, and understand why each pattern exists before moving to the next.

End of Document